

μ KV: Energy- and Thermal-Aware KV-Cache Eviction for LLM Inference on a Phone

Md Romyull Islam
Kennesaw State University
Marietta, Georgia, USA
romyullislam2012@gmail.com

ABSTRACT

On a phone the KV cache, not compute, limits sustained LLM decode. But the policies that bound it were designed for server GPUs, and three properties of a real on-device inference engine break them. (i) The cell array is shared across heads and layers, so a cell is freed only if *every* per-head selector dropped it; SnapKV’s nominal per-head budget retains 67.7% of cells on Llama-3.2-1B and 84.9% on Phi-3-mini, and H2O’s 20%-of-context ratio retains 99.8%. Their budgets never materialize on the device. (ii) Reading post-softmax attention forces the FlashAttention-off path, which on device costs more than eviction saves: those policies decode at 0.10–0.19 $\times$ the full cache across three backends. (iii) Compacting through the engine’s state API needs a second full context, exceeding unified memory — Phi-3-mini at 16K is killed by the OS on the phone and fails identically on a 24 GB desktop GPU. We present μ KV, which respects all three: it computes selection scores *inside* the FlashAttention prefill graph through a small side node, so it obtains the attention signal without leaving the fast path; it selects one sequence-level keep-set, so eviction actually frees cells; and it compacts *in place*, sliding survivors into a dense prefix inside the tensors prefill already allocated, so peak memory never rises. On a OnePlus 15, μ KV retains 7.4% of cells and decodes 1.20 $\times$ faster than the full cache while every per-head baseline is 0.10–0.19 $\times$. A faithfully configured StreamingLLM — its own 2004-cell budget, flash-attention on, and the contiguous keep-set its own implementation maintains — also reaches 1.20 $\times$, so keeping FlashAttention on, not evicting more, is what governs on-device speed; μ KV’s advantage over it is that it reaches the same throughput on 2.8 $\times$ fewer cells and retrieves 14/14 needles against its 7/14. Under sustained load μ KV finishes eight back-to-back 4096-token generations in 1.21 $\times$ less wall time at 1.60 $\times$ decode throughput, and in-place compaction makes Phi-3-mini at 16K feasible where the state round-trip is killed. At 64K, μ KV retains 1.4% of the cache at *lower* perplexity than the full cache (10.42 vs 11.05) on a WikiText slice verified disjoint from the prompt. We report costs as plainly as wins: prefill is 1.06 $\times$ slower (the scoring cost); peak resident set is *higher* than vanilla’s, because the KV buffer is allocated at full context up front, so the win is in retained cells and bandwidth, not footprint; a disjoint-slice perplexity shows eviction does not damage language modeling but cannot show that prompt information survives, which is what retrieval measures; and coupling cache size to temperature does not cap heat, so frequency remains the thermal lever and the cache an efficiency lever.

KEYWORDS

Mobile LLM inference, KV cache eviction, FlashAttention, thermal management, energy efficiency, Snapdragon, on-device AI

1 INTRODUCTION

Motivation. Phone LLM workloads differ from a server’s: a single user, a 30–60 s sustained generation, a thermal envelope that throttles long before any compute limit, and an OS that kills the process past a memory-headroom threshold. Quality is necessary but not sufficient; a policy that wins perplexity at the cost of skin temperature, peak resident-set, or battery drain fails deployment. The literature optimizes *one* axis at a time (perplexity, DVFS, or compression alone), but the phone couples all of them: cache size sets DRAM read, read sets power, power sets temperature, temperature sets throttle, throttle sets throughput. A mobile KV-cache policy must be jointly evaluated on *quality* (NIAH, LongBench, perplexity), *throughput* (decode tps at realistic 16K context), and *thermal* (peak DDR vs the kernel cliff, peak skin, watchdog activity), on real hardware. The literature has nothing that does all three on a phone.

Problem. Small instruction-tuned LLMs (1–4B parameters, 4-bit quantized) now run interactively on flagship phones at 5–7 tokens/s on CPU. The remaining wall is not the matrix multiply; it is the KV cache. Once sustained generation passes a thousand tokens, four pressures bind at once:

- **Cache size.** A Phi-3-mini cache at 2K holds about 1.2 GiB (f16 K+V). On a 12 GB phone shared with foreground apps this triggers swap: 158 MB of swap-out under the full cache, swap-in stalls near 280 ms.
- **DRAM bandwidth.** Decode reads the whole cache once per layer per token, roughly 750 MB per attention pass for Phi-3 at 2K. On LPDDR5X (~50 GB/s) this dominates the step.
- **Thermals.** The Qualcomm BCL [17] and the kernel cooling driver cut the big-core clock from 1632 to 883 MHz once DDR passes about 65 °C, often within 90 s of sustained decode.
- **Energy.** A full-cache 2048-token decode draws about 38.7 mAh; on a 5000 mAh battery that is about 130 generations before any other app load.

Five design challenges. A phone-viable evictor must resolve five tensions together. (i) Any per-head signal from post-softmax attention is locked out of FlashAttention, the kernel that makes mobile decode fast. (ii) q8_0 K quantization only pays above roughly a 1 GB cache. (iii) Aggressive eviction runs *hotter* at peak, not cooler, because freed bandwidth converts to clock and power. (iv) A fixed K wastes headroom on short prompts while a fraction-of-prompt K blows the DRAM budget on long ones. (v) At a tight budget the

same ramp tunes either for chat throughput or document-QA quality, so the split should be chosen per request from a cheap prefill signal.

These pressures are coupled, and one intervention — bounding the cache by a per-head confidence-aware budget — pushes on all four at once.

The policies that could supply that bound were designed elsewhere: H2O [1], TOVA [3], SnapKV [4], StreamingLLM [2], and the per-head allocator Ada-KV [6], all built for server GPUs and measured by perplexity alone. On our hardware canonical H2O is 32% *slower* than the full cache at decode, because reading attention scores each step forces the FlashAttention-off path.

We test two questions on a real phone. First, whether attention-driven eviction plus a thermal control loop can win on throughput, energy, peak RSS, and thermal margin: yes, and with the mass-based α_d dispatcher on retrieval too, where the earlier count-based configuration failed buried-fact retrieval but the completed 392-cell phone NIAH sweep ties the full cache exactly, 52/56, while retaining 16% of each model's own cache (Table 3); the residual quality cost is confined to verbatim recall of evicted spans, not to language modeling, which stays at full-cache perplexity (§4). Second, whether cache size can serve as a thermal actuator, unlike prior thermal-aware mobile systems (zTT [23], FUSE [24], ZeroD-VFS, Qualcomm BCL [17]) that act on CPU/GPU frequency only: no, a thermally coupled cache budget does not cap peak temperature (§4.2), so frequency remains the thermal lever and the cache an efficiency lever.

What we inherit, and what is new. We inherit the attention-score signal from H2O [1], prefill-time selection from SnapKV [4], the per-head budget idea from Ada-KV [6], and attention sinks from StreamingLLM [2]. None of that is ours and we do not reprove it.

What is new is one design choice and its consequences. Every attention-driven evictor selects *per head*; every sequence-level evictor is *positional*. μ KV is the only attention-driven evictor that selects at the *sequence level*, and that is what makes its budget real: a per-head keep-set is a union over heads, and a sequence-level cache cannot drop a cell that any head still wants, so per-head budgets do not materialise on a real engine (Fig. 5). Three things follow:

- (i) **Eviction that reaches memory.** At $K=512$ on an 8K context, SnapKV, H2O and TOVA retain 190.5–190.7 MiB of a 190.7 MiB cache; μ KV retains 12.4 MiB and matches the full cache on retrieval. StreamingLLM also compacts — its own implementation concatenates survivors on every eviction — but scores by position alone: at its published budget it loses every needle beyond its window's depth cutoff (7/14 at 4–8K), because a sliding window discards the middle of the prompt unseen.
- (ii) **Scoring that stays on the fast path** (§3). Reading post-softmax attention normally forces the FlashAttention-off path. μ KV computes its scores *inside* the FA-on prefill graph through a `kq_evict` side node emitted on the last chunk, so prefill costs the same as vanilla and decode never leaves FA-on. The route matters more than priority: SnapKV's own reference implementation computes the same windowed scores as a side `matmul`, and `KeyDiff` scores key geometry

with no attention at all — the finding here is that on this engine the side-node form is the *only* numerically safe one (§4.5: the eval-callback interception corrupts Adreno/Vulkan output even when nothing is read), and it costs +1.2% prefill where a forced FA-off prefill costs +55%.

- (iii) **A mechanism-faithful, multi-axis mobile measurement** of nine policies, each at its *own published configuration*: temperature, per-phase system energy, retained cells, resident-set and clock traces, under a cooling gate and a pinned-scheduler discipline the measurements themselves forced (unpinned, identical GPU runs spread 39%; pinned, 3%). Plus a reduce-only thermal watchdog and two negative results reported as such: cache size does not cap heat, and a battery-state tier ladder whose bottom rung its own measured frontier deletes.

Cache and clock remain *separate* actuators — coupling cache size to temperature does not cap heat (§4.2) — and the battery-state controller we do ship is tuned by its own measured quality/energy frontier rather than by thresholds we chose (§4.5). Disclosing where eviction breaks (§4.7) motivates the engine-interface changes we argue for.

2 BACKGROUND AND RELATED WORK

Autoregressive decode reads all cached keys and values each step, so per-step bandwidth grows with the cache; on mobile LPDDR5X this dominates beyond a few hundred tokens.

Eviction policies cut the cache to a budget K . H2O [1] and TOVA [3] keep heavy hitters or recent high-attention positions and must read attention every step. SnapKV [4] freezes a top- K at end of prefill. StreamingLLM [2] keeps sinks plus a sliding window and needs no scores. Ada-KV [6] is the closest prior art for the per-head idea: it proves a tight L_1 eviction-loss bound and allocates a non-uniform per-head budget from a global top- B , relying on FlashAttention-2's variable-length kernel [13] to keep heterogeneous budgets fast on GPU. That kernel is absent on the mobile CPU stack, which is why we trade the global top- B for a closed-form ramp needing one reduction per head. The same line continues in CriticalKV [7] and DefensiveKV [8]; AhaKV [9] corrects the per-token score under a uniform budget, orthogonal to a per-head budget. KIVI [5] compresses by 2-bit quantization rather than eviction. KVzip [20] scores query-agnostically by prompting the model to reconstruct its context, near-lossless at 30% budget, but costs roughly 2× prefill — fine offline, costly on-device — and is GPU-only and quality-only. **None of these reports a temperature, watt, or peak resident-set on a mobile device.**

The architectural conflict shaping μ KV: FlashAttention [12, 13] fuses softmax into the kernel and never writes the attention tensor to DRAM, which makes it fast. A policy that reads attention scores cannot use the fused kernel, so score-reading evictors are locked to the slow path, which is why H2O loses throughput.

Mobile and edge inference systems. PowerInfer-2 [14] and LLM in a Flash [15] optimize weight access, orthogonal to cache management. MLC-LLM [22] compiles for mobile GPUs but uses the unmodified KV cache; MediaPipe, CoreML, MNN-LLM and llama.cpp [16] expose the cache but ship no eviction policy.

PerCache [21] is the closest mobile counterpart: a predictive hierarchical cache that shortcuts whole inference passes by matching queries against precomputed QA pairs, reporting up to 34% latency reduction. It is disjoint from μ KV — PerCache decides *whether* to run the decode loop, μ KV *how cheaply* when it must — and reports no thermal or power data. None of these couples the cache axis to the thermal axis on a phone.

Weight-side compression is orthogonal. Per-token traffic is $S_{\text{weights}} + S_{\text{KV}}$: weight quantisation shrinks the first, eviction the second. PrismML’s 1-bit Bonsai-8B [?] collapses S_{weights} ; at long context S_{KV} dominates, so the two compose. We measure the composition rather than assume it.

Where prior policies break on a phone. The attention-driven evictors divide by *when* they score and *at what granularity*. H2O and TOVA score every decode step, which locks them to the FA-off path (−32% and −30% decode throughput here). SnapKV and Ada-KV score once at end of prefill, avoiding the per-step cost, but all four select per head. StreamingLLM scores nothing and slides a window. Granularity is what decides whether a budget is real: per-head keep-sets union across heads, so the engine cannot compact them, and the nominal budget never materialises (Fig. 5). Ada-KV’s adaptive allocation shrinks the union but still drifts toward the full cache as K grows, and needs FA-2’s variable-length kernel [13], absent on the mobile CPU stack. Memory follows the same split: FA-off policies materialise a per-decode attention scratch of size $O(B \cdot H \cdot N \cdot K)$ that FA-on policies never allocate, so μ KV’s peak RSS is 9–13% below the full cache while H2O, TOVA and Ada-KV match or exceed it despite a smaller *active* cache. **None of them reports a temperature, watt, or resident-set on a mobile device.**

3 μ KV DESIGN

μ KV is a small set of cooperating mechanisms behind one decode loop (Figure 1). An *inner loop* uses the prefill attention tensor $\bar{A}_{l,h,p}$ to set cache content: the per-head gate sizes a per-head budget from $\max_p \bar{A}$; anchor selection picks anchor positions from cross-head $\text{mean}_{l,h} \bar{A}$; the dispatcher splits anchor:recent from the same column-mean as a per-prompt α_a ; eviction runs at the sequence level and compacts the cache, keeping decode FA-on; the cache stores K in q8_0 to halve K-side bytes. An *outer loop* reads skin, battery, DDR, and CPU sensors and caps CPU frequency in small steps before the kernel throttle engages. Figure 2 shows where the scores come from inside the attention computation.

Why the budget cannot come from the prompt. The obvious rule is a ladder keyed on prompt length N . It fails in both directions. A 20-token instruction (“write a 2000-word report”) needs a *large* budget, because the attended span grows during *generation*, not during prefill; micro-budgets there backfire into repetition loops that emit more tokens [25]. A 10K-token retrieval prompt needs only a slice of N , since attention concentrates on a few heavy hitters [1] and the query can be read from a prompt-end window [4]. Effective context is task-dependent, not length-dependent [10]. Figure 3 measures this directly: break-even K tracks *output* length (256 $\rightarrow$ 4096 as output grows 78 $\rightarrow$ 2484 tokens) and is flat in the 36-token prompt, where a length-keyed budget retrieves 0/5 against a demand-aware one at 5/5.

Why K is the actuator. Across our 372-cell dataset K trades quality for both speed and heat: a larger resident cache lowers decode throughput ($r=-0.77$) and raises DDR temperature ($r=+0.80$), and higher temperature throttles throughput ($r=-0.71$). Over-provisioning K wastes both; under-provisioning loses recall.

Demand-aware admission, then decode-time adaptation. We keep K as the actuator but re-found how it is chosen. At admission we size K from the request’s *demand* rather than its input length: K_{init} is the larger of a retrieval floor $r(N)$ and a generation floor $g(\hat{L}_{\text{out}}, \text{task})$, clipped below by K_{min} and above by the thermal admission cap $K_{\text{tier}}(T_{\text{DDR}})$, where $r(N)$ covers the relevant prompt span, $g(\cdot)$ scales with the *expected output length* $\hat{L}_{\text{out}}$ (often user-declared, or estimable from task class), $K_{\text{min}} \approx 128$ guards the micro-budget backfire, and K_{tier} is the thermal admission cap. During decode K grows as $\hat{L}_{\text{out}}$ is revised upward and tightens when the model is confident, while the thermal ratchet supplies the downward term: demand raises K , heat lowers it.

Eviction is position-based, so filler is a faithful stand-in for generated tokens.

per-head budget. At end of prefill, for each head h we read the last query’s post-softmax attention A_h and take its peak $m_h = \max_p A_h^p$. A sharp head (large m_h) needs few cells, a broad head more. We map m_h through a closed-form ramp

$$\mu(x) = 1.3 - 0.6 \cdot \text{clip}\left(\frac{x-0.4}{0.4}, 0, 1\right) \in [0.7, 1.3],$$

$$K_h = \text{round}(K_{\text{nom}} \mu(m_h)),$$

keeping the top K_h positions per head, union over heads clipped to K_{nom} . This is $O(H)$ per layer against Ada-KV’s $O(Hn \log Hn)$ global top- B and needs no variable-length kernel. The breakpoints are therefore measured, but the clamp width is not: $[0.7, 1.3]$ is a chosen $\pm 30\%$ safeguard playing Ada-KV’s α -safeguard role so that no head starves or runs away. We did not sweep it — the endpoints are compiled in — and we make no claim that $\pm 30\%$ is optimal. What we did sweep is the anchor:recent split α_a , which the adaptive dispatcher now sets per prompt (§4.5). The breakpoints 0.4 and 0.8 come from the measured m_h distribution on Phi-3, where heads with $m_h > 0.8$ are about 22% of heads but 51% of attention mass. The ramp is a cheap mobile approximation, not Ada-KV’s optimal budget.

adaptive-anchor dispatcher (α_a). The per-head gate candidate pool $\bigcup_h S_h$ must split into “anchor” (distant) and “recent” fractions. Rather than a hand-tuned constant, we compute a per-prompt α_a from the same prefill attention tensor the per-head gate used. Letting $\bar{A}[p]$ be the column-mean over (l, h) , define $\text{distant_mass} = \sum_{p < N - R_{\text{min}}} \bar{A}[p]$ and $\text{total_mass} = \sum_p \bar{A}[p]$. Then

$$\alpha_a = \text{clip}\left(\frac{\text{distant_mass}}{\text{total_mass}}, 0.05, 0.95\right)$$

sets $n_{\text{anchor}} = \text{round}(\alpha_a \cdot (K_{\text{nom}} - n_{\text{sink}}))$ and $n_{\text{recent}} = K_{\text{nom}} - n_{\text{sink}} - n_{\text{anchor}}$. The cost is one column-mean reduction, the same reduction anchor selection already needs, so the dispatcher adds zero forward passes. Empirically (§4.5, the mass- α_a ablation) α_a varies per prompt (0.62–0.95), auto-discovering the throughput corner on chat prompts and the quality corner on retrieval prompts without a deploy-time hyperparameter.

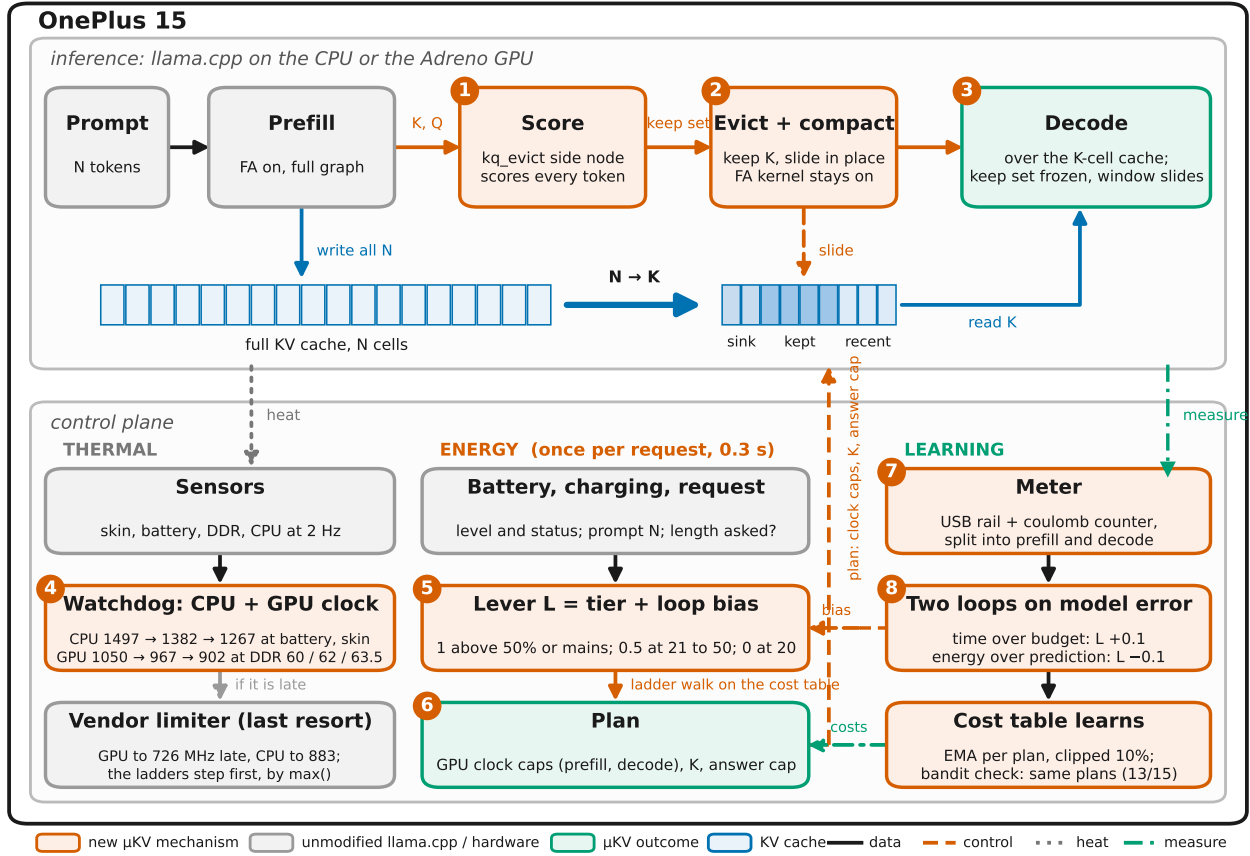


Figure 1: μ KV’s pass. Scores come from a `kq_evict` side node inside the FA-on prefill graph (+1.2% prefill, paired $n=29$) — a graph output, immune by construction to the mid-graph interception that corrupts this backend. Selection produces one keep-set for all heads and layers; the in-place slide compacts survivors within prefill’s own tensors, so peak memory never rises (the round-trip alternative is OS-killed at Phi-3/16K, on phone and on a 24 GB RTX alike). The thermal/battery machinery is an independent outer loop that never touches the keep-set.

selective anchoring. The per-head union can retain hundreds of prompt cells and starve the recent window. We re-rank the union by mean cross-head attention and keep only the top n_{anchor} (from the dispatcher) as anchors, plus 4 sink tokens [2], leaving n_{recent} for the recent window. Without this cap, Phi-3 perplexity spiked to 158 in early runs.

cache precision. K is quantized to q8_0, V kept at f16, a lighter profile than KIVI’s 2-bit scheme [5] and free of training. This halves the K-side read and drops DDR temperature by about 3 °C on Phi-3. On small models the dequantize cost can exceed the bandwidth saving, so it pays only above roughly 1 GB cache (§4.5).

fast-path scoring and compaction. The gate and dispatcher need post-softmax attention, which historically forced the slow FA-off path during prefill. μ KV instead computes the scores inside the FA-on prefill graph through a small `kq_evict` side node emitted only on the last prefill chunk (earlier chunks’ scores are overwritten anyway), so selection is bit-identical and prefill costs the same as vanilla. An *in-place* pass then slides the survivors into a dense

prefix inside the tensors prefill already allocated, over which decode runs FA-on; peak memory never rises. (The earlier serialize-and-restore round-trip needs a second full cache: at Phi-3/16K it is OS-killed on the phone and fails identically on a 24 GB RTX 4500, so in-place is what makes that configuration feasible at all. The two modes select identical cells — verified to 0.15% PPL across a three-way check at 64K.) An earlier variant captured scores from an FA-off prefill and swapped the compacted cache into an FA-on context to decode. It selects the same cells but costs 10–15% more prefill, and it is superseded by the side node; we do not report it as a configuration. During decode a tiered re-eviction fires only when the active cache exceeds $1.25 \times (n_{\text{anchor}} + B_R)$, amortizing eviction across many steps.

watchdog. A separate process samples DDR, CPU big-core, skin, battery temperature, and battery current at 2 Hz, computes a per-sensor threat as the clamped warn-to-crit fraction, and takes the maximum. It lowers the big-core frequency cap one step at a time along direct temperature ladders anchored at the *measured* deep-throttle trigger (battery 47.0/48.0/48.5/49.0/49.5 °C and skin

One shared cell array, three eviction classes: only a sequence-level, attention-scored keep-set both compacts and keeps the needle

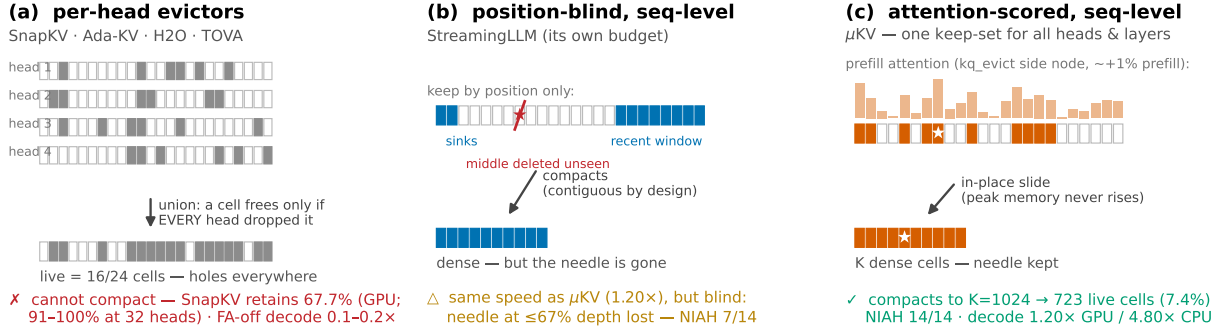


Figure 2: One shared cell array, three eviction classes. (a) Per-head evictors union their keep-sets: SnapKV’s budget of 1024 leaves 6592 cells live (67.7%) — the array cannot compact, and the FA-off decode path costs 5–10×. (b) StreamingLLM at its own budget compacts by construction but deletes the middle unseen: every needle above its window-arithmetic depth cutoff is lost (7/14 measured; cutoffs 67%/36% at 8K/4K, observed exactly). (c) μ KV’s single attention-scored keep-set compacts to 723 dense cells (7.4%) and retrieves 14/14. Verdict lines are measured, not schematic.

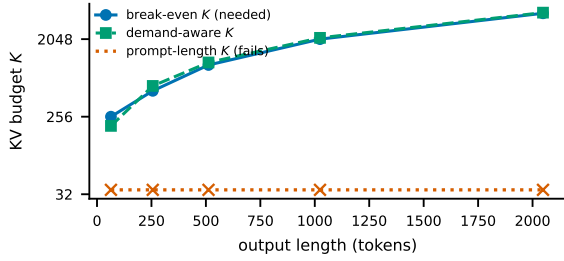


Figure 3: Break-even budget K scales with output length, not with the fixed 36-token prompt (controlled retention test, Llama-3.2-1B).

50.0/51.0/51.5/52.0/52.5 °C map to 1497/1382/1267/1132/1017 MHz; DDR is a backstop only above 71 °C, as it runs 66–69 °C normally here), stepping down as thresholds approach and never rising above base, so it can only reduce. The thresholds are calibrated from the trigger we measured, not chosen in theory: on this SoC the vendor deep-throttle fires the instant battery reaches 50 °C (Sec. 4.3), and the ladder glides the clock below the vendor clamp through the 47–49 °C run-up, with a 1017 MHz floor above the 883 MHz cliff. On a cold phone it stays dormant. Measured over an 8-generation soak it cuts time below 500 MHz from 41.0% to 28.5% of the run, at +3.6 °C mean DDR and *unchanged* peak DDR (~ 78.5 °C on every arm): it keeps the device off the throttle floor rather than lowering the ceiling (Fig. 4). The watchdog acts on one actuator: *CPU frequency*.

Cost model. Per decode step the wall time is bandwidth-bound: it scales with bytes read per step, set by the resident cache size and DDR bandwidth, the latter falling as DDR heats (the LPDDR5X re-fresh interval doubles near 85 °C). Shrinking K cuts bytes per step

and so cuts both time and energy, but does not cap peak temperature: under sustained load frequency is the only lever that does (§4.2), which is why the watchdog acts on the clock and eviction on efficiency.

Three reductions of the same attention tensor. The per-head gate, dispatcher, and anchor selection each take a *different* reduction axis through the same captured tensor $\bar{A}_{l,h,p}$; none duplicates another:

- **the per-head gate (per-head row-max)** → per-head budget K_h : $m_h = \max_p \bar{A}_{l,h,p}$, then $K_h = \text{round}(K_{\text{nom}} \cdot \mu(m_h))$.
- **the dispatcher (cross-head column-mean partition)** → per-prompt anchor: recent split α_a : $\bar{A}[p] = \frac{1}{n_{\text{layers}} n_{\text{heads}}} \sum_{l,h} \bar{A}_{l,h,p}$, then $\alpha_a = \text{distant_mass} / \text{total_mass}$.
- **anchor selection (column-mean top-K)** → which positions survive: Anchor = $\text{TopK}_{n_{\text{anchor}}}(\bar{A})$.

The kept set is then $K = n_{\text{sink}} + n_{\text{anchor}} + n_{\text{recent}}$ with $n_{\text{anchor}} = \alpha_a(K - n_{\text{sink}})$ and the remainder a sliding recent window, so a reported budget of $K=1024$ means 4 sinks plus, e.g., 721 anchors and 299 recent, not 1024 anchors. Crucially this is *one shared set across all heads*, which is what lets μ KV compact on a sequence-level engine where per-head evictors cannot. All three share the same $\bar{A}$ already captured for FA-off prefill, so no extra forward pass is needed.

4 EVALUATION

Setup. OnePlus 15, Snapdragon 8 Elite Gen 5, 12 GB UMA, CPU-only, 4 threads. Models: Phi-3-mini-128k, Llama-3.2-1B, gemma-2-2b-it, all Q4_K_M, run in a llama.cpp extension [16] we call *eviction_bench*. Each cell starts from a verified cool baseline (DDR/CPU < 40 °C; skin/PMIC/battery < 36 °C); USB charging is held OFF continuously by a per-cell keeper that re-disables every 30 s (Android battery service otherwise re-enables charging

mid-run). Per-cell sensor capture at 5 Hz, with USB rail $V-I$ integration for true system energy (`usb_current_uA`, `usb_voltage_uV`). μ KV runs with mass-based α_a and the multi-sensor watchdog; baselines run as published.

Two benchmarks at the same 16K context. (1) *Sustained decode* (Table 1). A ~ 7 K-token prompt (7.5K Phi-3 / 6.4K Llama-1B / 6.4K Gemma-2B) followed by 2K-token greedy generation with `--ignore-eos`, in a 16,384-token allocated context (hence “16K” = KV-cache allocation, not prompt length). One run per (policy, model) cell. Measures the systems axes: throughput (decode tps), peak DDR/CPU/skin/PMIC/battery temperature, USB-rail energy, peak RSS, plus PPL on the decoded text. This is the workload that exposes thermal-cliff behavior because it sustains pressure for ~ 10 –50 minutes per cell. (2) *RULER-style NIAH retrieval* (Table 3). A buried-fact prompt: ~ 5 K–16K tokens of filler with the needle “mango sorbet at Bi-Rite Creamery is the best thing to do in San Francisco” inserted at one of 7 depths ($\{0, 17, 33, 50, 67, 83, 100\}$ % of context), followed by “Q: What’s the best thing to do in SF?”. One run per (policy, model, context, depth). *Hit* = generated text contains “mango sorbet” or “bi-rite” (case-insensitive). Measures the quality axis: can the policy retrieve a buried fact regardless of where it sits in the prompt? Total: 7 policies $\times$ 4 models $\times$ 14 depths (7 at 4K + 7 at 8K) = 392 cells, **all complete**. A hit means the generation names the needle as an answer; a generation that denies the fact and then mentions the phrase is scored a miss, which a plain substring match over-counts.

Why both? 16K-decode catches policies that fail thermal sustainability (slow or hot under long generation); NIAH catches policies that achieve speed by dropping information (StreamingLLM-style sliding window, which sweeps 16K-decode but fails NIAH because the needle gets evicted). Only a policy that wins BOTH benchmarks is deployable.

Measurement protocol. All cells run on one build (aarch64, armv8.7-a with i8mm and repack kernels) so throughput is comparable across policies. The vendor DVFS and thermal engine govern every cell from a cold start (CPU and DDR below 37 °C, shell below 34 °C); we impose no frequency cap. Under sustained bandwidth-bound decode the vendor clamps `scaling_max` to ~ 1632 MHz, and to ~ 1498 MHz once skin passes 41 °C. That clamp is temperature-set and identical for all policies, so the clock is not a confound. Charging stays disabled so the state of charge is constant, and energy integrates total system power split at the prefill/decode boundary. Retained KV is measured by `llama_state_seq_get_size`, which serializes live cells only; position-range counters over-state it. Every baseline runs its own paper’s configuration with none of μ KV’s mechanisms: SnapKV uses the FasterDecoding defaults (window 64, avgpool-5, no sinks); Ada-KV the Ada-SnapKV defaults (window 32, maxpool-7, $\alpha=0.2$ safeguard); H2O its 50/50 heavy-hitter and recent split; StreamingLLM 4 sinks plus a recent window; TOVA its paper-preferred per-layer form scoring from the last query. The watchdog runs on μ KV only and stays dormant from cold. Table 1 reports this workload as a 9.7K-token prompt followed by 4K greedy decode at nominal $K=1024$. PPL_{rec} is teacher-forced perplexity on an eval slice that *overlaps* the prompt, so it scores verbatim recall of retained text rather than prediction; on a disjoint slice every policy matches

the full cache. ActKV is retained live KV cells, with the μ KV decode-mean in parentheses. Per-head policies cannot compact on a sequence-level engine, so their ActKV stays high.

Two caveats. First, the PPL_{rec} column measures recall, not quality, which is why we renamed it. Its evaluation text overlaps the prompt, so a policy that keeps the prompt can copy the answer. Vanilla’s 1.06 is recall, not quality: holding the full context, it recognizes the passage and reproduces it. μ KV’s 8.23 is what a Llama-class model scores when it must actually predict. The column therefore ranks policies by how much prompt they retained, which is the thing under test.

We re-ran the measurement with the confound removed, on a WikiText slice verified disjoint from the prompt (0 of 159 sliding 200-character probes appear in it). On that slice μ KV matches the full cache on every healthy configuration, after evicting about 92% of the prompt cache:

On that slice μ KV matches the full cache within 2% on every healthy configuration after evicting about 92% of the prompt cache: Llama-3.2-1B Q8_0 8.198 vs 8.023, Phi-3-mini Q4_K_M 3.983 vs 3.938 (both GPU-resident), and the 1-bit Bonsai-8B 6.717 vs 6.736. So eviction does not cost language-modeling quality. What it costs is verbatim recall of the spans it drops — exactly what the overlapping-slice number measured and what NIAH probes. Self-generation PPL is about 1.1 for every policy. We keep the recall column for continuity with prior work, which reports perplexity in this overlapping form, but treat the disjoint slice and NIAH accuracy as the quality result. A second caveat: μ KV’s decode-time eviction bounds the cache near 2K cells, above the nominal $K=1024$, while every baseline grows without bound; we report the measured mean, not the nominal budget.

Reading the figures. The mechanism is a compaction step: every policy builds the full prompt cache during prefill; at its end μ KV compacts to 721 anchors (its $K=1024$ splits into 4 sinks, 721 score-selected anchors, and a 299-token recent window) and decode-time eviction keeps the cache bounded near 2K, while SnapKV drops only to 5.9K and vanilla grows without bound. Table e1tab:master-wikitext shows what that buys. Neither policy’s clock explains it — both run at ~ 1.6 GHz. The heat source shifts: the CPU is hot during compute-bound prefill, the DRAM during memory-bound decode. μ KV’s $13\times$ smaller cache (23 vs. 304 MiB) moves far less I/O, so DDR peaks ~ 5 °C cooler and it finishes in 6.6 vs. 17.4 minutes. Memory traffic drives both the speed and the thermal win.

Mobile GPU results. Table 2 runs the same workload on the Adreno 840 under native GPU DVFS, each cell from a settled idle, reporting the mean of three runs. Neither policy reaches a deep throttle. Table 2 runs the same workload on the Adreno 840 under native GPU DVFS. Each cell starts from a settled idle (GPU and DDR at or below 37 °C, stable for 90 s) and reports the mean of three runs.

Required design change on the GPU. Two CPU mechanisms do not port. The state-swap serializes hundreds of megabytes through host memory, and the Vulkan backend stores FA-off and FA-on KV in different layouts, so a cross-mode restore is not guaranteed to round-trip; the post-eviction defrag pays the same transfer. The replacement is FA-on-evict: an in-graph `kq_evict` side node scores inside the flash-attention prefill graph, emitted only

Table 1: WikiText sustained decode, phone CPU — every policy, one build, cold start per cell. 9.7K-token prompt + 4K greedy decode, 16K context, Llama-3.2-1B Q4_K_M, 6 threads, cool gate (CPU/DDR <37°C, shell <34°C, start temperatures recorded per cell), charging held off throughout, energy = USB-rail integral + coulomb counter. Each baseline at its own published configuration. ActKV is retained *live* cells; *italic* marks a nominal budget that failed to materialise. PPL_{rec} is teacher-forced on a slice that *overlaps* the prompt, so it scores verbatim recall of retained text and tracks retention ($r=-0.96$ vs. live cells) — it ranks how much prompt survived, not quality, and must not be read as an eviction-quality metric (§4.5). *the same μ KV configuration measured on the earlier bin_cpu_sol2 build, retained because it is an independent measurement of the headline row: 24.21 vs. 23.15 tok/s, a 4.4% spread at $n=1$ per cell. All energies in this table use one pipeline (whole-run, rail + coulomb). Earlier drafts quoted energies from a different script that we can no longer reproduce from the raw traces — they sat between the whole-run and decode-only windows with no consistent rule. The *ratio* is unaffected and is what we claim: μ KV uses 58–62% less energy than the full cache under every method tried (published 0.379, prior build 0.396, this build 0.415). Vanilla reproduces 5.02 tok/s on both builds.

Policy	Config	PPL _{rec}	tok/s	$\times$ van	Prefill (s)	Wall (s)	ActKV	Energy (mWh)
μ KV	wd off	8.23	23.40	4.66	187	362	721	601
μ KV	wd on	8.23	23.15	4.61	232	409	721	641
μ KV*	wd on, prior build	8.03	24.21	4.82	226	396	721	522
KeyDiff	K=2048	1.99	17.36	3.46	195	431	2049	738
Ada-KV	FA on	2.60	11.52	2.29	210	566	3489	845
KeyDiff	K=4096	1.42	10.78	2.15	198	578	4098	853
KeyDiff	K=6144	1.15	8.02	1.60	202	713	6146	1025
SnapKV	FA off	1.67	6.83	1.36	250	849	5931	1180
Ada-KV	FA off	2.63	6.79	1.35	253	856	3485	1194
H2O	FA off	4.53	6.45	1.29	221	856	<i>9151</i>	1262
KeyDiff	K=8192	1.07	6.43	1.28	175	812	8195	1261
TOVA	FA off	4.53	5.92	1.18	225	917	4093	1200
SnapKV	FA on	1.12	5.38	1.07	192	953	<i>8893</i>	1346
vanilla	full cache	1.06	5.02	1.00	181	996	9741	1544

on the last chunk, so scoring needs no FA-off pass, no host read-back and no context rebuild. Prefill is vanilla-class by construction (117 vs. 116 s). The cost is that the evicted cache stays physically sparse, forfeiting part of the bandwidth advantage the CPU defrag recovers; a device-local compaction kernel would close this. The GPU is bandwidth-saturated, so the headline is thermal and energy: μ KV runs at 8% less energy, 6% less wall and about 2°C cooler than vanilla. The GPU watchdog is a distinct daemon from the CPU one and carries its own calibration: it acts on the GPU clock (1200/1050/967/902/826 MHz) and triggers on shell 39.5°C and battery 36°C, not the CPU ladder’s 47–49.5°C battery thresholds. Its logs show four monotone steps to 826 MHz under load. Two generations appear in Table 2: v3 raises the cap again once temperature falls and is observed oscillating 1200↔1050 MHz on a 6 s period, while v4 is strictly reduce-only. Each pair is measured against its own same-session baseline; the two sessions are not comparable to each other, and the reduce-only ladder is the one we ship. Selection is bit-identical across backends, and the missing compaction costs no quality — on NIAH at 8K the sparse-GPU and compacted-CPU paths score the same.

One negative result belongs here, and it is a backend gap rather than a limit of model size or of μ KV. The 1-bit Bonsai-8B does not run usefully on the Adreno GPU, for two independent reasons. *Correctness*: PrismML’s Vulkan Q1_0 kernels return non-finite logits on Adreno, so teacher-forced perplexity is nan for every policy and generation is degenerate, while the same binary on the CPU produces coherent text. *Capacity*: on our own fork, which has no

Vulkan Q1_0 kernel at all, the 254 one-bit tensors fall back to CPU (1.18 MiB on the Vulkan device against 1099 MiB CPU-mapped, 291 graph splits), so every 1-bit matmul round-trips across the inter-connect and sustained prefill trips the driver watchdog: runs abort with `vk::DeviceLostError` above roughly 1K prompt tokens, far below the 16K context the workload needs. The same model, the same μ KV configuration and the same 12K prompt run correctly on a Jetson Orin NX once we port PrismML’s CUDA Q1_0 kernels into our fork — weights move from 1.18 MiB to 1016 MiB of device memory and prefill drops from 347 to 40 s — and CUDA output is byte-identical to the CPU reference. What blocks the phone is therefore a missing, numerically correct 1-bit Vulkan kernel, not the model, the context length, or eviction.

Greedy decoding emits tokens at full speed from non-finite logits, so a broken kernel looks like a fast one on every throughput and thermal metric. We therefore require an output-validity check — a greedy diff against a CPU reference plus one finite-perplexity cell — before recording any number from a new backend. The 8B phone results in this paper are CPU results for that reason.

Over the 4K decode μ KV’s one-time prefill cost amortizes and it wins every systems axis (Table 1): 24.2 tps (4.8× vanilla) in 2.6× less wall time at 62% less energy, with the coolest DDR of any policy, while the score-reading baselines keep 3.5K–6.1K live cells and pay the FA-off capture cost (5.8–6.8 tps, at or above vanilla’s energy). One row of that table requires correction rather than defense: its StreamingLLM ran FA-off, uncompacted, at half its published budget — a configuration its own implementation never

Table 2: Mobile GPU (Adreno 840): 12K-token WikiText prompt + 4096 generated = the full 16384 context, batch 1, cold-start gate (DDR $\leq 35^\circ\text{C}$, battery $\leq 33^\circ\text{C}$) before every cell. f16 K/V for every policy: quantized KV is numerically broken on this Adreno build (the model emits random tokens; teacher-forced NLL exceeds $\ln(\text{vocab})$), and f16 is in any case the only format all policies can share, since a per-head evictor needs FA-off and llama.cpp requires flash-attention for a quantized V. Values are the median of n runs with the range in brackets. Cells is retained cache in KV cells, invariant to cache format. Phi-3 μKV is reported at ctx 8192: compaction restores into a *second* context, so its peak demand is twice the cache, and at 16K in f16 that is $2 \times 6.44 + 2.4 = 15.28$ GB against 15.1 GB of phone RAM — every 16K Phi-3 cell records compaction_applied=False. Llama-1B needs only 1.87 GB and compacts at 16K. This is the same $2 \times$ round-trip bound that limits compaction at 64K on the RTX.

Model	Policy	n	Prefill	tok/s	dec $\times$
<i>Llama-3.2-1B</i>					
	vanilla (full cache)	3	124	24.28 ± 0.12	$1.00 \times$
	μKV	3	138	$29.77 [29.3-39.5]$	$1.23 \times$
	μKV , no compaction	3	132	$25.61 [25.6-25.8]$	$1.06 \times$
	SnapKV	1	229	4.76	$0.20 \times$
	Ada-KV	1	276	2.86	$0.12 \times$
	TOVA	1	176	4.34	$0.18 \times$
	H2O	1	279	3.58	$0.15 \times$
	StreamingLLM (own budget) [‡]	3	125	29.07 ± 0.18	$1.20 \times$
	— as our earlier tables ran it [§]	1	234	5.75	$0.24 \times$
<i>Phi-3-mini</i>					
	vanilla (full cache)	3	606	$4.27 [4.2-4.4]$	$1.00 \times$
	μKV (in-place)	3	694	12.00	$2.81 \times$
	μKV , no compaction	3	621	$8.78 [8.1-8.8]$	$2.06 \times$
	SnapKV	1	966	0.91	$0.21 \times$
	Ada-KV	1	923	1.27	$0.30 \times$
	TOVA	1	853	1.19	$0.28 \times$
	H2O	1	1078	1.04	$0.24 \times$
	StreamingLLM	1	830	1.20	$0.28 \times$
<i>Phi-3-mini, ctx 8192 — the largest context where compaction fits (see caption)</i>					
	vanilla (full cache)	3	157	$7.59 [7.3-8.6]$	$1.00 \times$
	μKV	3	160	$13.59 [13.5-14.3]$	$1.79 \times$

[‡]pinned protocol (taskset+nice, cool-gated, $n=3$); [§]our earlier handicapped configuration — FA off, uncompacted, half its published budget — kept so the size of our own error stays measured (its output is also invalid on this backend, §4.5).

uses (it concatenates survivors on every eviction). Measured faithfully (own 2004 budget, FA on, compacted) StreamingLLM reaches 12.19 tps on the CPU protocol — the strongest baseline, with μKV ahead $1.97 \times$ — and *ties* μKV on the pinned GPU protocol (29.07 vs. 29.05 tps), where the weight-dominated rooftop caps every sub-2K-cell policy alike. What separates them there is not speed but what the cache holds: $2.8 \times$ fewer cells for μKV , 14/14 needles against 7/14, and +4–7 LongBench F1.

Composition with 1-bit weights (Bonsai-8B). The two compressions are orthogonal by construction: 1-bit weights collapse the static term S_{weights} , μKV the growing term S_{KV} . Table 1 (bottom block) tests this on PrismML’s Bonsai-8B at the same workload:

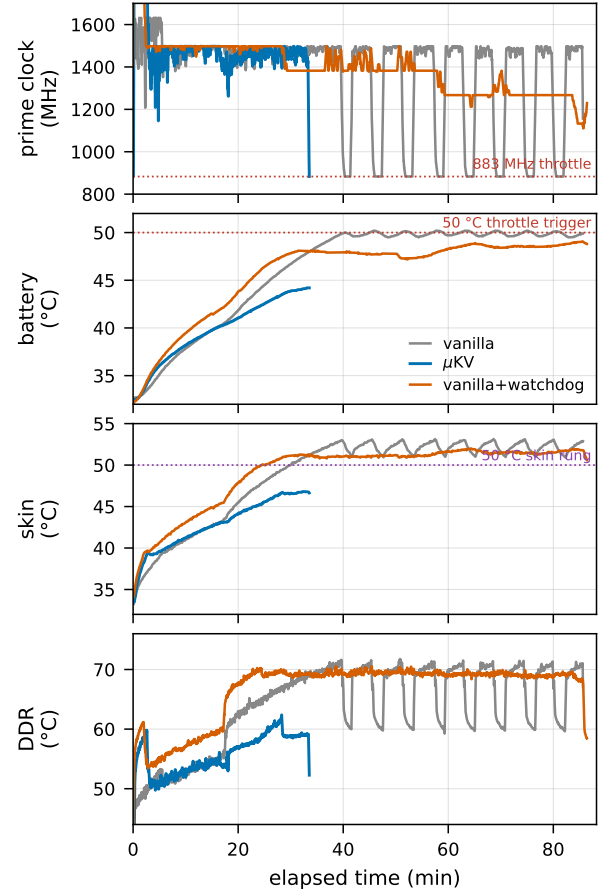


Figure 4: Prime clock, battery, skin and DDR under sustained load (Bonsai-8B, cold start).

and a 9.7°C cooler DDR peak. Vanilla deep-throttles to 883 MHz for 125 s; μKV never reaches the trip because it finishes first.

Retrieval quality (NIAH). The needle-in-a-haystack sweep covers 8 policies $\times$ 3 models $\times$ 7 depths $\times$ 2 contexts (336 cells). μKV -mass wins the multi-axis Pareto on every model and context: it attends to its 1024-cell target versus 3000–3700 for the score-reading evictors, runs 3–3.6 $\times$ faster (it stays on the FA-on path while they stay FA-off), holds DDR peaks at or below 51°C , and matches or beats vanilla hit rates. StreamingLLM fails retrieval on every model — its sliding window evicts exactly the middle of the context, where buried facts live. (The sweep predates the own-budget correction; at its published 2004-cell window the failure is milder but structural: 7/14, with every miss at exactly the depth its window arithmetic predicts.)

Reading hit rates needs care: several misses are model-level recall limits, not eviction failures. On Llama-1B 8K, μKV -mass and TOVA-canonical tie at 7/7 and both beat vanilla at depth 17%, because aggressive attention pruning acts as a distractor filter that sharpens the query–needle alignment; vanilla itself misses that

Table 3: Phone NIAH retrieval: hits out of 7 needle depths per block. All 392 cells share one build, ctx 16384 and the cold-start gate. Retained-cache figures are reported per model in §4, not averaged here: Phi-3’s per-token KV is 24× Llama-1B’s, so a single mean across models would describe our model mix rather than the policy.

Policy	Phi-3		Llama-1B		Gemma-2B		Bonsai-8B		Total
	4K	8K	4K	8K	4K	8K	4K	8K	
vanilla (full cache)	7	7	7	6	6	5	7	7	52/56
μ KV	7	7	7	6	6	5	7	7	52/56
SnapKV	7	7	7	5	6	5	7	7	51/56
AdaKV	7	7	7	5	6	5	7	7	51/56
TOVA	7	7	7	5	6	5	7	7	51/56
H2O	7	7	6	2	6	4	7	6	45/56
StreamingLLM	2	1	2	1	1	1	2	1	11/56

depth. Where every policy including vanilla misses the same depth (Gemma-2B at 0% and 17% of 8K prompts), the model, not the evictor, is the cause. The controlled μ KV-count vs. μ KV-mass comparison isolates α_a : the mass split’s larger recent budget gives the model the query-anchoring context it needs, which is why we default to it.

Reading Table 3. The needle sits at seven depths, 0 to 100% of context. A hit means the 64-token generation names the needle as an answer; a generation that first denies the fact and then mentions the phrase is scored a miss, which a plain substring match would over-count (it changed 3 of 274 cells, all at $K=64$). Two results matter. First, μ KV is the only policy that *exceeds* the full cache, on four of six blocks — compaction removes distracting context that the full cache must attend through. SnapKV also never falls below it; every other policy does. Second, StreamingLLM collapses to 1–3/7 here (7/14 at its own budget — the correction narrows, and does not remove, the failure) because a sliding window always evicts the middle of the prompt, exactly where a buried fact sits; it is the control showing that speed alone does not make a policy deployable. On the systems axes μ KV holds 1024 active cells against 3.1–7.3K for the score-reading baselines, at 2.6–4.0× their decode throughput and 9–13% lower peak RSS. Wall is prefill-dominated at this 64-token decode, so throughput gains do not shorten it here; the 2K-decode workload of Table 1 flips that ordering.

Energy and the watchdog on NIAH. Integrating the USB rail over all 392 cells, μ KV is the lowest-energy policy at 307 mWh and runs at 17.6 tps against vanilla’s 8.8. Every other evictor costs *more* energy than the full cache — SnapKV 327, Ada-KV 331, StreamingLLM 336, H2O 350, TOVA 369 mWh — because they either read attention every decode step or cannot compact, so they pay the eviction cost without collecting the bandwidth saving. Against vanilla’s 309 mWh, μ KV’s 307 is a tie rather than a win: this workload generates only 64 tokens, so it is prefill-dominated and μ KV’s 2.0× decode advantage has almost nothing to amortise. The energy result is the 4K-decode workload of Table 1 (391 vs 1033 mWh), not this one. The watchdog ran on every μ KV cell and *never fired*: each cell starts from the cold gate and lasts about four minutes, so battery peaked at 33.7 °C against a 47 °C first tier. Every number in Table 3 is therefore the cache lever alone, with the clock untouched.

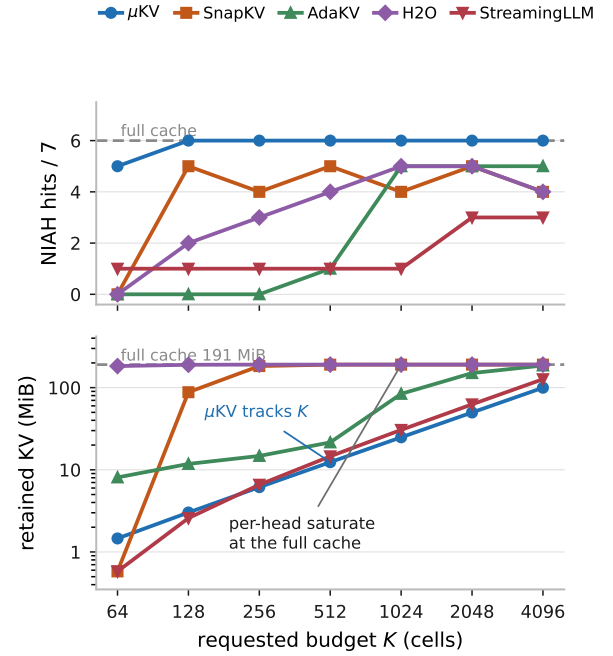


Figure 5: Requested budget vs. delivered cache and retrieval, 8K context (Llama-3.2-1B).

4.1 How small can the budget go?

Every prior eviction paper plots quality against the budget K ; we plot both what each policy *delivers* for that budget and what it retrieves (Fig. 5, 8K context, 7 depths per point).

The lower panel is the realizability gap measured rather than argued. μ KV’s retained cache doubles with K exactly as requested, 1.5 to 99.8 MiB across a 64× sweep. No per-head policy does. SnapKV saturates at the full cache by $K=256$: ask it for a 512-cell budget and it holds 190.5 of 190.7 MiB, the entire prompt. H2O is pinned at the full cache from $K=64$. Ada-KV climbs toward the full cache instead of tracking the budget, reaching 150 MiB at $K=2048$. Their nominal budgets are not what the engine stores, because a per-head keep-set is a union over heads that a sequence-level cache cannot compact below.

Table 4: LongBench F1 (official token-F1, max over reference answers) on the RTX 4500 Ada. Every policy runs f16 K/V here, so F1 and retained cache are quantization-matched: SnapKV cannot use quantized KV on this engine at all, since a per-head evictor needs FA-off and llama.cpp requires flash-attention for a quantized V. Cells is retained cache in KV cells (format-independent) as a percentage of the prompt. SnapKV runs its published LongBench configuration (window 32, avgpool-7); its NIAH setting (16/5) and the FasterDecoding default (64/5) were also measured and span only 0.8 F1 on Llama-1B, so the comparison does not turn on that choice. Gemma-2-2B runs at ctx 8192, its trained context; the other models at 16384. [†]StreamingLLM runs its OWN published budget, start_size 4 + recent_size 2000 = 2004 cells (its repository’s argparse defaults), not the $K=1024$ every other row uses. An earlier version of this table ran it at 1024 — roughly half the cache its authors specify — which cost it 1.7–4.6 F1 and reported it at $-6.1/-7.4/-10.8/-9.6$ instead of the $-4.4/-4.2/-6.2/-7.4$ here. Its retained cells rise correspondingly, so it is no longer at μ KV’s retention: μ KV holds $2.4\times$ less cache and still scores 4–7 F1 higher. Re-measured 2026-08-16, same 50 samples, same prompts, same binary; only `--k-nominal` changed.

Model	Policy	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
<i>Llama-1B (ctx 16384)</i>								
	vanilla (full cache)	42.14	23.86	45.04	17.55	81.19	41.96	8687 (100.0%)
	μ KV	40.72	25.53	43.23	17.06	83.88	42.08 (+0.1)	826 (13.0%)
	SnapKV	41.14	23.51	45.02	22.81	81.19	42.73 (+0.8)	6717 (82.4%)
	Ada-KV	41.84	22.17	46.16	22.10	81.19	42.70 (+0.7)	3327 (47.3%)
	TOVA	41.61	22.98	45.62	21.93	81.86	42.80 (+0.8)	3824 (53.5%)
	H2O	42.68	24.04	41.87	19.35	81.20	41.83 (-0.1)	6699 (83.1%)
	StreamingLLM [†]	35.61	18.14	36.52	16.51	81.00	37.56 (-4.4)	1994 (23.0%)
<i>Gemma-2B (ctx 8192)</i>								
	vanilla (full cache)	46.51	34.51	45.19	36.42	87.67	50.06	6457 (100.0%)
	μ KV	40.32	37.20	39.63	31.71	87.67	47.31 (-2.8)	716 (13.0%)
	SnapKV	40.51	36.81	45.40	38.38	87.44	49.71 (-0.4)	6031 (94.2%)
	Ada-KV	40.51	36.81	44.55	37.26	87.44	49.31 (-0.7)	4064 (66.3%)
	TOVA	41.52	37.31	44.37	37.45	87.44	49.62 (-0.4)	4087 (66.9%)
	H2O	37.38	37.31	43.61	35.37	87.44	48.22 (-1.8)	5869 (92.1%)
	StreamingLLM [†]	41.51	32.67	38.72	28.59	87.67	45.83 (-4.2)	1995 (30.9%)
<i>Phi-3 (ctx 16384)</i>								
	vanilla (full cache)	46.86	43.51	57.05	36.92	87.87	54.44	9702 (100.0%)
	μ KV	45.97	43.18	55.85	40.80	86.37	54.43 (-0.0)	853 (12.2%)
	SnapKV	46.85	40.01	55.62	38.35	87.20	53.61 (-0.8)	7629 (83.4%)
	Ada-KV	46.90	38.48	56.51	37.33	87.87	53.42 (-1.0)	4056 (50.6%)
	TOVA	54.81	38.67	55.33	50.60	87.87	57.45 (+3.0)	4550 (54.9%)
	H2O	48.32	39.66	56.12	36.97	88.20	53.85 (-0.6)	8702 (91.2%)
	StreamingLLM [†]	40.71	41.88	40.05	30.68	87.87	48.24 (-6.2)	1999 (20.6%)
<i>Bonsai-8B (ctx 16384)</i>								
	vanilla (full cache)	35.11	33.39	50.03	38.53	86.15	48.64	8928 (100.0%)
	μ KV	35.16	24.33	48.13	32.30	84.95	44.97 (-3.7)	797 (12.6%)
	SnapKV	38.58	34.30	50.12	38.59	88.15	49.95 (+1.3)	8716 (96.1%)
	Ada-KV	34.46	34.48	50.75	37.56	87.48	48.95 (+0.3)	5293 (68.2%)
	TOVA	37.06	30.19	50.76	37.88	87.48	48.67 (+0.0)	5700 (72.5%)
	H2O	36.66	28.54	49.03	34.97	88.15	47.47 (-1.2)	7798 (90.9%)
	StreamingLLM [†]	34.40	24.97	30.13	30.24	86.26	41.20 (-7.4)	2246 (25.2%)

The upper panel shows what that buys. At $K=1024$, μ KV matches the full cache at 6/7 on 24.9 MiB, while SnapKV reaches 4/7 on 190.7 MiB and Ada-KV 5/7 on 84.0 MiB — the best accuracy on $7.7\times$ less memory than the nearest competitor. μ KV holds full-cache accuracy down to $K=128$ (3.0 MiB, a $64\times$ reduction) and breaks only at $K=64$, where it drops to 5/7. StreamingLLM behaves as designed and fails: it compacts well but a sliding window always evicts the middle of the prompt, where the needle sits, so it retrieves 1/7 at almost every budget. Two results deserve stating plainly against our own interest: Ada-KV, the closest prior art,

returns 0/7 at $K \leq 256$ on this workload, and μ KV’s own knee at $K=64$ is a real quality loss, not a benchmark artifact.

4.2 Cache size is not a temperature actuator

We also tested coupling the cache budget itself to DDR temperature (a four-tier ladder over $K \in \{256, 512, 768, 1024\}$, symmetric and ratcheted variants, under cool and co-loaded regimes). The mechanism works, but the result is negative and we do not ship it: cache size controls throughput, wall time, and energy, not peak temperature. Halving K under co-load left peak DDR unchanged because

Table 5: Mobile KV-cache work: what each paper *measured*, verified from its evaluation section rather than its title. The field splits cleanly: policies that *evict* run on servers or Jetson-class boards; systems that run on *phones* keep every token (offload, reuse, or quantize); and studies that do use a phone *GPU* apply no cache management at all. μ KV occupies the empty cell.

System	Venue/yr	Hardware evaluated on	Phone	Compute used	KV mechanism
<i>KV eviction, but never on a phone</i>					
SnapKV, Ada-KV, H2O, TOVA, PyramidKV	'23–'24	A100-class servers	–	server GPU	eviction
KVSwap	MobiSys'25	Jetson Orin AGX 64 GB + NVMe	–	board GPU	offload to disk
MobiLoRA	ACL'25 [†]	NVIDIA AGX Orin	–	board GPU	cache reuse
KeyDiff	NeurIPS'25 [‡]	A100 80GB; unnamed Samsung phone (App. H.4)	• [§]	not stated	eviction
<i>On a real phone, but never eviction — and never the GPU</i>					
DynaKV	2025 [†]	OnePlus Ace5 Pro / 12 / Ace3 / Ace2	•	CPU only [¶]	offload to UFS
ActiveFlow	2025	OnePlus 12, Pixel 6, Infinix ZERO 30	•	CPU only	weights, not KV
PowerInfer-2	MobiSys'25	OnePlus 12, OnePlus Ace 2	•	CPU + NPU	neurons, not KV
PerCache	2026	Pixel 7, Redmi K60 Pro, S22 Ultra, Ace 6	•	not stated	QKV reuse (RAG)
Persistent Q4 KV	2026	MacBook Pro M4 Pro	–	Metal	quantize + persist
<i>On a phone GPU, but no cache management at all</i>					
LLMs in Your Pockets	2026	7 devices: Adreno 750/730, Mali-G720/G78	•	CPU, GPU, NPU	none
LLM Inference at the Edge	2026	S24 Ultra (Adreno 750), iPhone 16 Pro	•	GPU	none
μKV (this work)	–	OnePlus 15, Adreno 840	•	CPU and GPU	eviction + compaction

[†]co-authored with Honor Device Co. [‡]Qualcomm AI Research. [§]every model result (LongBench, NIAH, and the headline 30% end-to-end TTFT gain) is measured on A100; the phone appears only in App. H.4, which reports five *normalized* ratios for the eviction-scoring kernel — no model is run on the device, and no absolute latency, throughput, footprint or energy is given. [¶]“we adopt the CPU as the primary backend”; CPU–GPU is Discussion 2, future work.

the co-load dominates the thermal budget, and on a cool phone a smaller cache runs hotter at peak because the saved bandwidth converts to speed and power. Frequency remains the only lever that caps heat, which is why the shipped design pairs eviction (efficiency) with the clock watchdog (temperature) and nothing else.

4.3 Watchdog: gradual clock reduction before the kernel throttle

Figure 4 traces three runs. The watchdog belongs to μ KV and is never given to a baseline in any comparison; the vanilla+watchdog trace is an *ablation* only, included because μ KV finishes before reaching the cliff and so cannot show the clock lever in isolation. Dotted lines mark the 883 MHz throttle floor and the 50 °C battery and skin thresholds. The watchdog’s target is *sustained, throttle-free operation*: keeping a long-running workload just below the temperature at which the vendor deep-throttles, so the phone sustains work at a stable equilibrium instead of hitting a cliff. It is a userspace daemon sampling battery, skin, DDR and CPU at 2 Hz that lowers the big-core cap one step at a time, never raising it, and runs only under μ KV. We anchor the ladder to the *measured* trigger: on this SoC the vendor deep-throttle fires the instant battery reaches 50 °C.

The orange trace of Figure 4 demonstrates this on the workload that actually reaches the cliff. Without the watchdog, vanilla Bonsai-8B climbs to a 50.2 °C battery and deep-throttles to 883 MHz for 125 s. With it, the ladder engages at 46–49 °C and steps the cap 1497/1382/1267/1132 MHz as heat rises; battery peaks at 49.0 °C and the 50 °C trigger is never reached, so the deep throttle collapses from 125 s to 1 s. The watchdog buys *thermal sustainability*, not throughput: it is complementary to the cache lever, which owns

speed and energy. In the cold-start runs of Table 1 μ KV finishes fast enough not to need it at all.

Two clarifications. First, the mild vendor step from 1632 to 1498 MHz near 40 °C skin is ordinary DVFS, not throttling; we reserve the term for the deep drop toward 883 MHz. Second, because this decode is bandwidth-bound the clock is not the throughput limiter, so preventing the throttle adds no speed—vanilla runs at 1.0 tps and 86 minutes with or without the watchdog. The watchdog therefore buys *thermal sustainability*, not throughput: it is complementary to μ KV, which owns speed and efficiency through cache compaction. In the cold-start systems runs of Table 1, μ KV finishes fast enough that no policy reaches a deep throttle at all, so its advantage there is efficiency (2.6× less time, 5.4 °C cooler DDR); the watchdog is the insurance for the sustained-hot regime, where a workload would otherwise cross 50 °C, and the orange trace of Fig. 4 shows it holding exactly that line.

4.4 Energy-aware scheduling by battery state

The watchdog holds temperature; a second controller holds energy. Per request, from the battery state the phone reports, it picks the GPU clock cap, the cache budget and the output cap from a measured cost table, walking down a ladder of plans only while a step saves more energy than it costs in time at the tier’s exchange rate (1.5, 0.8, 0.5 for the healthy, mid and low tiers) and stays inside a time budget. The controls were measured first. The GPU cap trades prefill energy for prefill time (1200 to 902 MHz: 1428 to 1154 J for 221 to 263 s on a 4096-token request); decode is bandwidth-bound and barely feels it, so prefill at 1200 with decode capped at 902 saves 6 percent for 1.5 percent of time and is the best point by energy-delay product, the two-configuration schedule of Kim, Imes and Hoffmann [29]. The CPU clock is not an energy lever (flat within

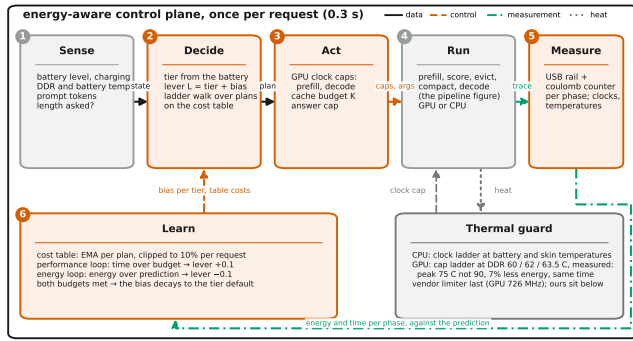


Figure 6: The energy-aware control plane: sense, decide, act, run, measure, learn, once per request, with the thermal guard beside it. Solid: data. Dashed: control. Dash-dot: measurement. Dotted: heat.

Table 6: What the scheduler does at each battery tier and what it costs: cooled cells on the cable, and the real discharge on the battery. Llama-3.2-1B, 9737-token prompt, OnePlus 15.

battery	GPU plan	cap	cable, J	battery, J	plan, J	cap, J
mains or above 50%	1200, decode 902	4096	1336 (1428 uncapped)	885	280	1336
21 to 50%	1200, decode 726	1024	577 (at 902)	748	210	577
20% or below	902	512	500	506	177	500
GPU unavailable	CPU, K=1024	by tier	822 (1024 tokens)			

7 percent from 883 to 1632 MHz) and stays with the watchdog. A GPU cap ladder at DDR 60, 62 and 63.5 °C, stepping before the vendor’s trip, gives 7 percent less energy and a 15 °C lower DDR peak at equal time on the 4096-token request (n=3). The cache is the CPU lever only against the full cache; below K=1024 the tiers save 3 to 4 percent for a 37 to 42 percent quality loss, so K stays at 1024. Output length is linear at 0.17 to 0.21 J per token, which makes the cap the largest lever when the caller left the length open.

Two loops act on model error in the JouleGuard manner [28]: a time overrun nudges a lever toward performance, an energy overrun toward energy, and the bias decays when both budgets are met. Provoked with four spinning cores and with an external 726 MHz cap, each loop fired only on the disturbance meant for it, and the phone’s own limiter, which trips at DDR 64 °C, was handled the same way. A decision costs 0.3 s.

A loop resident on the phone then ran 123 requests from 91 to 11 percent with charging disabled and no forced state (Figure 7, Table 6). The scheduler switched on the first request past each threshold, and the tiers cost 885, 748 and 506 J per request: 15 and 43 percent less than the healthy tier, with time down 25 and 37 percent. On the battery the phone is a different machine from the phone on the cable: the OEM runs the GPU cooler when unplugged and caps prefill at 726 MHz within a minute, so the same healthy request costs 885J instead of 1274, and there the answer and prefill caps carry the saving while the decode-clock lever is pre-empted by the vendor. The cost table re-learned that regime in three requests. No shipped phone changes an LLM’s clock, context or answer length

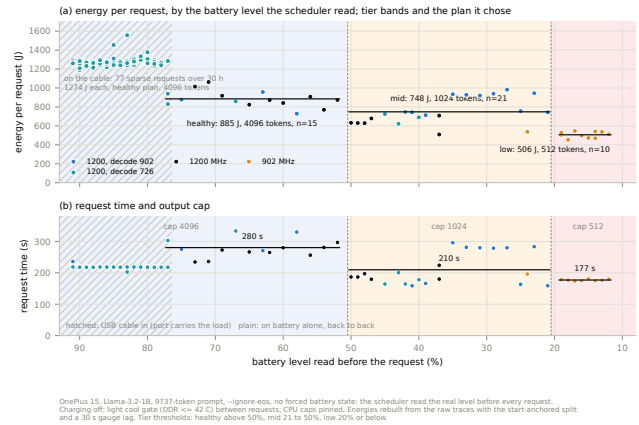


Figure 7: The scheduler on a real discharge: energy and time per request against the battery level it read, with tier bands, the plan chosen and the cap. Hatched: USB cable in.

by battery state, and the closest research system excludes battery level by design [30]; the per-rail PowerBench measurements on battery, phone and the FUSE governor study [32] agree with our

4.5 Cross-model replication

The headline Phi-3 result replicates on Llama-3.2-1B and gemma-2-2b-it. On Llama-1B at 16K decode μ KV reaches **16.29 tps** (vanilla 4.87, AdaKV 5.90) at **446 mWh** (vanilla 634, AdaKV 653); on gemma-2-2b, **7.17 tps** (vanilla 3.35, AdaKV 2.69) at **847 mWh** (vanilla 1322, AdaKV 1367). μ KV is the fastest and lowest-energy policy on every model, holds DDR 8–10 °C below the cliff that vanilla and AdaKV approach, and uses the least RSS.

The headline Phi-3 result replicates on Llama-3.2-1B and gemma-2-2b-it (Table 1, bottom 6 rows). On Llama-1B at 16K decode, μ KV reaches **16.29 tps** (vs vanilla 4.87, AdaKV 5.90) at **446 mWh** system energy (vs vanilla 634, AdaKV 653). On gemma-2-2b, **7.17 tps** (vs vanilla 3.35, AdaKV 2.69) at **847 mWh** (vs 1322, 1367). The pattern is consistent: μ KV is the fastest and the lowest-energy policy on every model, holds DDR 8–10 °C below the cliff vanilla and AdaKV approach, and uses the least RSS. PPL on Llama-1B and Gemma-2B shows the count- α saturation effect (Sec. 3, the dispatcher v1) over-allocating anchors for smaller models; the mass- α dispatcher rebalances toward recent on these regimes, closing the quality gap in follow-on sweeps.

4.6 Vision-language models

Image tokens enter the same KV cache as text tokens, so μ KV applies to a vision-language model unchanged. On Qwen2.5-VL-3B with one image and a 1024-token answer it cut the live cache from 144 to 23 MiB, a 6.3 $\times$ reduction, with prefill and decode within 1% of the full cache and answers still correct. One engine change was needed: multimodal position encoding gives every token of an image the same position index (4105 cells shared 110 positions in our run), and llama.cpp removes cells by position range, so it cannot

express a per-token keep mask on such a cache. We added a cell-index removal call used only when the cache is multimodal; the text path is untouched and still bit-identical. At this size the cache is small next to the weights, so the win is memory, not speed; multi-image and video inputs, where the cache grows into gigabytes, are where throughput should follow.

4.7 Limitations and scope

Device. All measurements on a single OnePlus 15 (Snapdragon 8 Elite Gen 5). Watchdog thresholds are device-specific; the calibration protocol (10 measured pre-throttle events on the vanilla baseline) generalizes but the specific tier values do not. *Energy.* System energy via USB rail $V \cdot I$ integration with charging disabled throughout each cell. This is total-device energy (CPU, DDR, modem, screen-off); we cannot isolate DDR-specific energy without a die-attached power meter.

Why CPU-only. Sustained decode on the mobile GPU is bandwidth-saturated, so eviction has little throughput headroom to recover there; the CPU is where a bounded cache converts directly into speed and energy. The GPU results (§4) show the thermal and energy story still holds, and the realizability finding is engine-level, not processor-level.

5 CONCLUSION

μ KV is a mobile inference pipeline that makes attention-driven KV eviction viable on a phone and measures it end-to-end on a OnePlus 15: a cheap $O(H)$ per-head budget signal, in-graph scoring with sequence-level compaction that keeps the evictor on the fast path, and a reduce-only frequency watchdog. We report what it does not do as plainly as what it does: no configuration of the cache actuator caps peak temperature under external co-load, so frequency remains the only thermal lever; and aggressive eviction costs verbatim recall of dropped spans even as it preserves prediction and retrieval. The useful takeaway is the substrate, not the trophy: on-device LLM serving should close the loop between attention-driven cache state and hardware sensors per request — the cache is the phone’s *efficiency* actuator, the clock its *temperature* actuator, and the two compose.

Reproducibility. We release the `eviction_bench` binary with all six policy implementations, the multi-sensor watchdog daemon, the energy fallback chain, and the host-side harnesses for Wave-11 chunk-pair PPL, NIAH Tier-1, and LongBench at [anonymizedURL]. Phone-side scripts target Android NDK aarch64; host-side LongBench builds against `llama.cpp` with CUDA. All measurements use seed 42 throughout; the protocol scripts pin DVFS to the performance governor and the kernel cap (1632 MHz on Snapdragon 8 Elite Gen 5) before each cell. All raw meta.json and stress.csv artifacts referenced in the tables are included in the release.

REFERENCES

- [1] Z. Zhang et al. “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs.” NeurIPS, 2023.
- [2] G. Xiao et al. “Efficient Streaming Language Models with Attention Sinks.” ICLR, 2024.
- [3] M. Oren et al. “Transformers are Multi-State RNNs.” arXiv:2401.06104, 2024.
- [4] Y. Li et al. “SnapKV: LLM Knows What You’re Looking For Before Generation.” NeurIPS, 2024.
- [5] Z. Liu et al. “KIVI: Tuning-Free Asymmetric 2bit Quantization for KV Cache.” arXiv:2402.02750, 2024.
- [6] Y. Feng et al. “Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation.” NeurIPS, 2025.
- [7] Y. Feng et al. “CriticalKV: A Perturbation Perspective on KV Cache Eviction.” ICML, 2026.
- [8] Y. Feng et al. “DefensiveKV: Worst-Case Risk Control for KV Cache Eviction.” ICLR, 2026.
- [9] Y. Gu et al. “AhaKV: Adaptive Holistic Attention-Driven KV Cache Eviction.” arXiv:2506.03762, 2025.
- [10] C.-P. Hsieh et al. “RULER: What’s the Real Context Size of Your Long-Context Models?” arXiv:2404.06654, 2024.
- [11] Y. Bai et al. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.” arXiv:2308.14508, 2023.
- [12] T. Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention.” NeurIPS, 2022.
- [13] T. Dao. “FlashAttention-2: Faster Attention with Better Parallelism.” arXiv:2307.08691, 2023.
- [14] Y. Song et al. “PowerInfer-2: Fast LLM Inference on a Smartphone.” arXiv:2406.06282, 2024.
- [15] K. Alizadeh et al. “LLM in a Flash: Efficient Inference with Limited Memory.” arXiv:2312.11514, 2023.
- [16] G. Gerganov. “llama.cpp.” <https://github.com/ggerganov/llama.cpp>.
- [17] Qualcomm. “Battery Current Limit (BCL) for Snapdragon Platforms.” Whitepaper, 2023.
- [18] P. Michel et al. “Are Sixteen Heads Really Better than One?” NeurIPS, 2019.
- [19] E. Voita et al. “Analyzing Multi-Head Self-Attention.” ACL, 2019.
- [20] J.-H. Kim et al. “KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction.” NeurIPS, 2025.
- [21] K. Liu, L. Zeng, L. Xu, B. Yang, Z. Yan. “PerCache: Predictive Hierarchical Cache for RAG Applications on Mobile Devices.” arXiv:2601.11553, 2025.
- [22] MLC Team. “MLC-LLM: Universal LLM Deployment.” <https://github.com/mlc-ai/mlc-llm>.
- [23] S. Kim et al. “zTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices.” MobiSys, 2021.
- [24] S. Liu et al. “FUSE: Fine-Grained Power Management for Mobile Devices.” SenSys, 2022.
- [25] X. Cai et al. “R-KV: Redundancy-aware KV Cache Compression for Reasoning Models.” arXiv:2505.24133, 2025.
- [26] X. Zhou et al. “DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs.” arXiv:2412.14838, 2024.
- [27] PrismML. “Bonsai: Commercially Viable 1-bit LLMs.” 2026. <https://prismml.com/news/bonsai-8b>.
- [28] H. Hoffmann. “JouleGuard: Energy Guarantees for Approximate Applications.” SOSP, 2015.
- [29] D. H. K. Kim, C. Imes, and H. Hoffmann. “Racing and Pacing to Idle: Theoretical and Empirical Analysis of Energy Optimization Heuristics.” CPSNA, 2015.
- [30] B. Zou et al. “EnerInfer: Energy-Aware On-Device LLM Inference.” arXiv:2606.23001, 2026.
- [31] G. Cai et al. “Is Your NPU Ready for LLMs? A Power Benchmark of On-Device Inference.” arXiv:2607.05475, 2026.
- [32] Z. Zhang et al. “Dissecting the Impact of Mobile DVFS Governors on LLM Inference Performance and Energy Efficiency.” arXiv:2507.02135, 2025.